DEVOPS & CLOUD ENGINEERING | AWS PRODUCTION | LIVE STATUS: VERIFIED

SPANQC Infrastructure & Deployment Playbook

Comprehensive Technical Handover & Operational Runbook for Cloud Infrastructure, Containerized Microservices, Automated Provisioning, and CI/CD on Amazon Web Services.

Document Purpose & Audience

This document is prepared exclusively for DevOps Engineers, Site Reliability Engineers (SREs), and Cloud Architects taking ownership of the SPANQC AWS infrastructure, provisioning pipelines, container orchestration, secrets management, and day-2 operations.

TARGET APPLICATION **SPANQC (Tech Pub Wiring Diagram QC System)**	CLOUD PROVIDER & REGION **Amazon Web Services (AWS) — ap-south-1 (Mumbai)**
LIVE ELASTIC IP / INGRESS URL **3.109.239.116 (http://3.109.239.116)**	INFRASTRUCTURE AS CODE **Terraform (~> 5.0) & Ansible (Core 2.15+)**
HOST SPECIFICATIONS **AWS EC2 t3.medium (Ubuntu 24.04 LTS, 4GB Swap)**	DEPLOYMENT STACK **Docker CE, Docker Compose v2, Next.js 16, Python 3.12**

DOCUMENT CONTENTS AT A GLANCE

1. **Section 1:** Cloud & Container Topology (with High-Resolution Vector Architecture Diagram)
2. **Section 2:** DevOps Tech Stack, Tooling Matrix & Repository File Layout
3. **Section 3:** Step-by-Step AWS Infrastructure Provisioning with Terraform
4. **Section 4:** Step-by-Step Host Hardening & Configuration Management with Ansible
5. **Section 5:** Containerization & Microservice Orchestration (Docker Compose)
6. **Section 6:** Production DevOps Battle-Tested Gotchas & Technical Fixes
7. **Section 7:** Secrets Matrix, Environment Variables & Day-2 Operational Runbook

SPANQC Engineering © 2026. All Rights Reserved.

1. Cloud & Container Architecture

SPANQC is deployed as a resilient, single-host microservices topology inside a dedicated AWS Virtual Private Cloud (VPC). The architecture cleanly isolates public web delivery, background job processing, and AI transcription / deterministic verification.

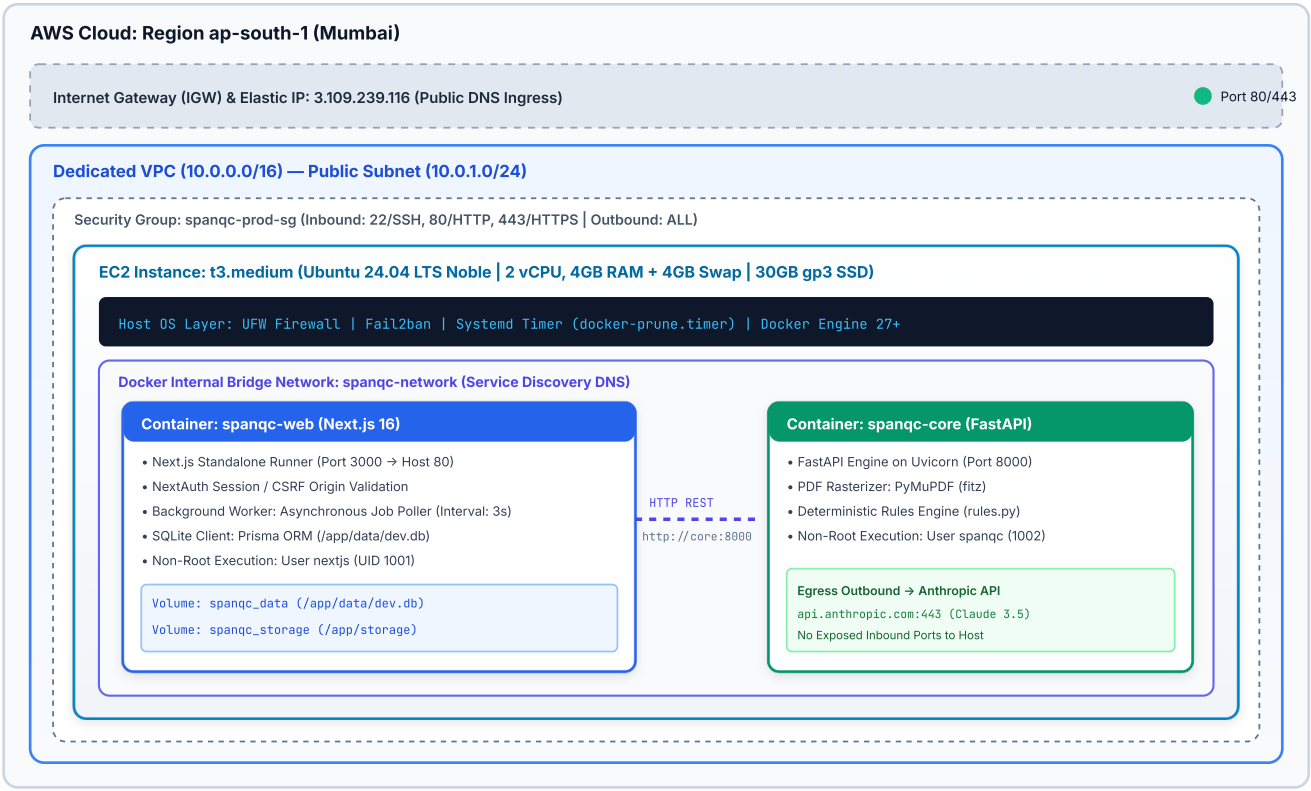


Figure 1.1: Complete End-to-End Infrastructure, Network, and Microservices Topology.

1.1 Architectural Principles

- **Deterministic Verification with AI Transcription:** Anthropic Claude Vision transcribes drawing metadata, pin configurations, and wire IDs; `spanqc_core/rules.py` executes 100% deterministic code-level verification without LLM hallucination.
- **Fail-Closed Security Posture:** Missing keys, partial files, or schema corruptions yield `REVIEW` or structured errors. Never simulated, never false passes.
- **Isolated Microservices Networking:** The Python verification engine (`spanqc-core`) has zero exposed external ports. It is reachable exclusively by the Next.js gateway via the private Docker bridge network (`http://core:8000`).
- **Zero Overhead Data Storage:** Production utilizes high-speed SQLite on persistent Docker named volumes with WAL (Write-Ahead Logging) mode, avoiding multi-hundred dollar RDS overhead for initial high-performance throughput.

2. Tooling Matrix & AWS Provisioning (Terraform)

The infrastructure was engineered with industry-standard Infrastructure as Code (IaC) and configuration management tools to guarantee idempotent, reproducible deployments.

Category	Technology	Version	Role / Configuration in SPANQC
Cloud Provider	Amazon Web Services	ap-south-1	VPC, IGW, Route Tables, Security Group, EC2 Instance, Elastic IP, gp3 Storage.
IaC	Terraform	>= 1.5.0	Provisions all AWS networking and computing assets with dynamic inventory output.
Config Management	Ansible Core	>= 2.15	Configures base OS, SWAP, firewall, docker daemon, and deploys microservices.
Host OS	Ubuntu Server	24.04 LTS (Noble)	Base operating system with hardened kernel parameters, UFW firewall, and Fail2ban.
Container Engine	Docker CE	27.x + Compose v2	Multi-container orchestration, volume persistence, and isolated bridge network.
Frontend & API	Next.js (Standalone)	16.3.6 (Turbopack)	Node.js 20 Alpine runner, reverse proxy handler, NextAuth, Prisma SQLite engine.
Core QC Service	FastAPI / Python	3.12-slim	Uvicorn ASGI server, PyMuPDF (fitz) rasterizer, Anthropic Claude Vision API client.

2.1 Step-by-Step AWS Provisioning (Terraform)

The AWS foundation is declared in `infra/terraform/`. It constructs an isolated, secure perimeter with a static Elastic IP:

Step 1: Authenticate & Initialize Terraform

```
# Authenticate and navigate to terraform directory
export AWS_DEFAULT_REGION="ap-south-1"
cd /data/projects/uiux/infra/terraform

# Download AWS provider plugins (⇝ 5.0) and validate
terraform init -upgrade
terraform validate
```

Step 2: Plan and Provision AWS Assets

```
# Generate execution plan and apply infrastructure
terraform plan -out=tfplan
terraform apply -auto-approve tfplan

# Inspect generated outputs
terraform output
# public_ip  = "3.109.239.116"
# instance_id = "i-09ab7..."
# Dynamic inventory automatically written to: infra/ansible/inventory.ini
```

Step 3: Terraform Managed Resources Matrix

- **aws_vpc (main):** Dedicated CIDR block `10.0.0.0/16` with DNS hostnames and DNS resolution enabled.
- **aws_subnet (public):** Subnet `10.0.1.0/24` in AZ `ap-south-1a` with public IP allocation on launch.
- **aws_security_group (spanqc-prod-sg):** Port 22 (SSH), Port 80 (HTTP Web), Port 443 (HTTPS), Egress (All).
- **aws_instance (web):** `t3.medium` instance running Ubuntu 24.04 LTS with 30GB gp3 encrypted root volume.
- **aws_eip (spanqc_ip):** Static public Elastic IP (`3.109.239.116`) associated with the EC2 instance.

> **DevOps Safety Check**
> Do NOT delete or release the Elastic IP in the AWS Web Console. The IP `3.109.239.116` is mapped directly in customer DNS configurations. Always manage life cycles through Terraform.

3. Host Hardening & Configuration (Ansible)

Once Terraform provisions the EC2 instance, Ansible takes over to configure the server, apply security benchmarks, install the container runtime, and deploy the application.

3.1 Ansible Roles Breakdown

1. Role: common

- Creates a **4GB swapfile** at `/swapfile` to absorb high-concurrency Node.js build spikes on t3.medium.
- Tunes `vm.swappiness = 10` and increases `fs.inotify.max_user_watches`.
- Configures **UFW Firewall**: Default deny incoming, allow 22/tcp, 80/tcp, 443/tcp.
- Deploys **Fail2ban** with SSH jail to deter automated brute-force attacks.

2. Role: docker

- Configures the official Docker APT repository with verified GPG key.
- Installs `docker-ce`, `docker-ce-cli`, `containerd.io`, and `docker-compose-plugin`.
- Adds deployment user (`ubuntu`) to the `docker` group for passwordless container management.
- Enables and starts the `docker.service` systemd daemon.

3. Role: maintenance

- Deploys `docker-prune.service` (oneshot `docker system prune -af`).
- Deploys `docker-prune.timer` running every Sunday at 03:00 AM.
- Cleans dangling images older than 168 hours to prevent EBS storage exhaustion.

4. Role: spanqc_app

- Creates application directories at `/opt/spanqc`.
- Synchronizes application code with strict exclusions (no `node_modules`, `.git`, or `.terraform`).
- Generates production `.env` file from Jinja2 template.
- Executes `docker compose up -d --build` with orphan removal.

3.2 Step-by-Step Ansible Execution Guide

Step 1: Test Server Connectivity

```
cd /data/projects/uiux/infra/ansible

# Ping remote host via SSH using dynamic inventory
ansible -i inventory.ini spanqc_servers -m ping
```

Step 2: Run Full Provisioning Playbook

```
# Execute all roles with secrets passed safely via extra-vars
ansible-playbook -i inventory.ini playbook.yml \
  --extra-vars "anthropic_api_key=$ANTHROPIC_API_KEY auth_secret=$AUTH_SECRET"
```

Step 3: Targeted Tag Execution (For Rapid Updates)

```
# Deploy application code changes only (skipping OS hardening)
ansible-playbook -i inventory.ini playbook.yml --tags "app,deploy"

# Re-configure firewall and base packages only
ansible-playbook -i inventory.ini playbook.yml --tags "common"
```

3.3 Automated Maintenance Systemd Timer

To eliminate disk leaks from Docker build caches on production servers, Ansible sets up an automated systemd timer:

```
# Check active status of Docker pruning timer on host
ssh -i ~/.ssh/id_ed25519 ubuntu@3.109.239.116 "systemctl status docker-prune.timer"
```

4. Containerization & Service Orchestration

SPANQC services run as non-root, isolated Linux containers governed by `docker-compose.yml`.

4.1 Multi-Stage Dockerfile Architecture

Frontend Container (Dockerfile.web)

- **Stage 1 (deps):** Installs dependencies with `npm ci`, runs `npx prisma generate`.
- **Stage 2 (builder):** Builds standalone Next.js server with `NODE_OPTIONS="--max-old-space-size=1024"`.
- **Stage 3 (runner):** Minimal Alpine Linux runner (approx. 180MB total size). Runs as unprivileged user `nextjs:nodejs` (UID 1001).
- **Entrypoint:** `docker-entrypoint.sh` creates persistent volume directories and initializes SQLite schemas.

Core Engine Container (Dockerfile.core)

- **Base:** `python:3.12-slim` with minimal runtime utilities.
- **User:** Runs as unprivileged user `spanqc:spanqc` (UID 1002).
- **Components:** FastAPI, PyMuPDF (C-based high-speed vector rasterizer), Anthropic SDK, and deterministic rule validation engine.
- **Healthcheck:** Actively checks `curl -f http://127.0.0.1:8000/health` every 20 seconds.

4.2 Production Docker Compose Specification

```
services:
  web:
    container_name: spanqc-web
    build: { context: ., dockerfile: Dockerfile.web }
    image: spanqc-web:latest
    restart: unless-stopped
    ports:
      - "80:3000"                          # Direct host port 80 mapping
    environment:
      - NODE_ENV=production
      - DATABASE_URL=file:/app/data/dev.db
      - AUTH_SECRET=${AUTH_SECRET}
      - PYTHON_SERVICE_URL=http://core:8000 # Internal Docker DNS
    volumes:
      - spanqc_data:/app/data              # Persistent SQLite database
      - spanqc_storage:/app/storage        # Uploaded diagram PDFs and crops
    depends_on:
      core:
        condition: service_healthy         # Waits until core healthcheck passes
    networks:
      - spanqc-network

  core:
    container_name: spanqc-core
    build: { context: ., dockerfile: Dockerfile.core }
    image: spanqc-core:latest
    restart: unless-stopped
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - SPANQC_MODEL=claude-sonnet-4-6
    networks:
      - spanqc-network                     # No exposed host ports (Internal only)

volumes:
  spanqc_data:
  spanqc_storage:

networks:
  spanqc-network:
    driver: bridge
```

5. Real-World DevOps Gotchas & Fixes

During the production deployment of SPANQC, five critical infrastructure and networking challenges were encountered and permanently resolved. Incoming engineers must understand these solutions:

Gotcha #1: Docker Build Context Out-of-Memory (OOM)

Symptom: Running `docker compose build` on EC2 took over 12 minutes or crashed with exit code 137 (OOMKilled).

Root Cause: The build context included the `infra/terraform/` directory, which contained heavy Terraform provider binaries (`.terraform/` = 872 MB) and temporary archives.

Permanent Fix: Updated `.dockerignore` to explicitly exclude `infra/`, `experiments/`, and `.git/`. Reduced context transfer from **872 MB down to 30 kB**. Builds now complete in under 45 seconds.

Gotcha #2: Next.js 16 Reverse Proxy 10MB Body Truncation

Symptom: Uploading large wiring diagram PDFs or multi-sheet ZIP archives failed with `400 Bad Request: Malformed multipart form data`.

Root Cause: Next.js 16 Turbopack introduced a strict 10 MB default payload cap on server proxy routes.

Permanent Fix: Added `experimental: { proxyClientMaxBodySize: "100mb" }` in `next.config.ts`. The server now comfortably parses high-resolution 50MB+ engineering CAD exports.

Gotcha #3: CSRF Mismatch on Direct Host Port 80 Access

Symptom: Login or Register forms failed with `Invalid origin` or CSRF rejection when accessing via the public IP `http://3.109.239.116`.

Root Cause: The container listens internally on port 3000 while the host listens on port 80. Standard `request.url` parsed host as `localhost:3000`, mismatching the browser's `Origin: http://3.109.239.116`.

Permanent Fix: Modified `src/lib/server/auth/csrf.ts` to prioritize `request.headers.get("host")` over internal URL hostname.

Gotcha #4: Inter-Service Docker DNS Discovery

Symptom: Web container logs showed `ECONNREFUSED 127.0.0.1:8000` when dispatching QC jobs to the Python engine.

Root Cause: In Docker bridge networks, `localhost` inside container A refers to container A, not container B.

Permanent Fix: Configured `PYTHON_SERVICE_URL=http://core:8000` and `SPANQC_PYTHON_SERVICE_URL=http://core:8000`. Docker's embedded DNS automatically resolves `core` to the Python container's IP.

Gotcha #5: Database Schema Push Inside Container

Symptom: Adding new tables to `prisma/schema.prisma` did not immediately reflect in the running database on EC2.

Solution: Run the migration command inside the running container: `docker compose exec web npx prisma db push`. This updates `/app/data/dev.db` on the persistent volume without requiring a service restart.

6. Secrets Matrix & Day-2 Operations Runbook

6.1 Production Environment Variables Reference

Variable Name	Container	Description & Value Format
`DATABASE_URL`	spanqc-web	Path to SQLite database: `file:/app/data/dev.db`
`AUTH_SECRET`	spanqc-web	32+ character cryptographic secret for session JWT encryption.
`NEXTAUTH_URL`	spanqc-web	Public origin: `http://3.109.239.116` (or customer domain).
`PYTHON_SERVICE_URL`	spanqc-web	Internal microservice route: `http://core:8000`
`ANTHROPIC_API_KEY`	spanqc-core	Anthropic Claude API key (Format: `sk-ant-api03-...`).
`SPANQC_MODEL`	spanqc-core	Model target: `claude-sonnet-4-6`
`ADMIN_EMAILS`	spanqc-web	Comma-separated list of administrative tenant accounts.

6.2 Day-2 Operational Commands (Cheat Sheet)

1. Inspect Live Container Logs

```
# Tail all container logs simultaneously
docker compose -f /opt/spanqc/docker-compose.yml logs -f

# Tail web application logs only
docker compose -f /opt/spanqc/docker-compose.yml logs -f web

# Tail Python validation engine logs only
docker compose -f /opt/spanqc/docker-compose.yml logs -f core
```

2. Hot-Deploy Code Updates

```
# On EC2 Host (/opt/spanqc):
git pull origin main
docker compose build
docker compose up -d
```

3. Database Backup & Snapshot

```
# Create safe hot backup of SQLite database while application is live
docker compose exec web sqlite3 /app/data/dev.db ".backup '/app/data/backup_$(date +%Y%m%d_%H%M%S).db'"

# Copy backup to host filesystem
docker cp spanqc-web:/app/data/backup_*.db /home/ubuntu/backups/
```

4. System Health & Resource Verification

```
# Verify container CPU and Memory usage
docker stats --no-stream

# Check systemd cleanup timer status
systemctl status docker-prune.timer
```

> **End of Playbook**
> For emergency support or architecture questions, refer to the source repository at `https://github.com/Adhavan07/Spanqc` or contact the primary DevOps maintainer.